

Evolución versiones Java

Versión 8 (2014)

Se abren las puertas a la programación funcional con el uso de expresiones Lambda y Streams. Se realiza una revisión de APIS y se actualiza de forma importante la gestión de fechas. Todo esto lo hablarán otros compañeros.

Versión 13 (2019)

Incluye las siguientes características nuevas, así como cientos de mejoras menores y miles de correcciones de errores.

- Dynamic CDS Archives
- ZGC: Uncommit Unused Memory
- Reimplement the Legacy Socket API
- Switch Expressions (Preview)
- Text Blocks (Preview)

Destaca: **NullPointerException** (NPE)

NullPointerException (NPE)

NullPointerException es una excepción que indica que la aplicación intentó usar una referencia a un objeto que tenía un valor nulo. Extiende la clase RuntimeException, que a su vez incluye excepciones que se lanzan en tiempo de ejecución. Además, NPE es un unchecked exception, por lo que no es necesario manejarlo y el compilador no informa un error en tiempo de compilación. Las unchecked exceptions representan fallas en el código creado por el programador (sí, es nuestra culpa). Por lo general, estos son errores de aplicación que podrían haberse evitado si el desarrollador hubiera tenido más cuidado al programar.

Versión 14 (2020)

Incluye las siguientes características nuevas, así como cientos de mejoras menores y miles de correcciones de errores.

- Pattern Matching for instanceof (Preview)
- Packaging Tool (Incubator)
- JFR Event Streaming
- Non-Volatile Mapped Byte Buffers
- Records (Preview)
- Deprecate the Solaris and SPARC Ports
- Remove the Concurrent Mark Sweep (CMS) Garbage Collector
- Deprecate the ParallelScavenge + SerialOld GC Combination
- Remove the Pack200 Tools and API
- Text Blocks (Second Preview)
- Foreign-Memory Access API (Incubator)

Destaca: **Switch statements** (Lo explica otro compañero)

Versión 15 (2020)

Añade, por ejemplo, soporte para literales de cadena multilínea (también conocidos como bloques de texto). Los recolectores de basura Shenandoah y Z (este último a veces abreviado como ZGC). Se elimina el soporte para el sistema operativo Solaris de Oracle. Se elimina el motor JavaScript Nashorn. También se eliminan algunos certificados raíz de CA.

- Edwards-Curve Digital Signature Algorithm (EdDSA)
- Sealed Classes (Preview)
- Hidden Classes
- Remove the Nashorn JavaScript Engine
- Reimplement the Legacy DatagramSocket API
- Disable and Deprecate Biased Locking
- Pattern Matching for instanceof (Second Preview)
- ZGC: A Scalable Low-Latency Garbage Collector
- Text Blocks
- Shenandoah: A Low-Pause-Time Garbage Collector
- Remove the Solaris and SPARC Ports
- Foreign-Memory Access API (Second Incubator)
- Records (Second Preview)
- Deprecate RMI Activation for Removal

Destaca: **Garbage collector, text blocks**

Garbage collector

Un Garbage collector (recolector de basura) es como un ayudante automático que se encarga de borrar de la memoria las cosas que tu programa ya no está usando.

Piensa en esto:

- Tu programa crea objetos, datos, listas, etc.
- Algunos los usas un rato y luego ya no los necesitas.
- Si nadie los borra, se quedan ocupando espacio para siempre.

El recolector de basura entra en acción y dice:

“Oye, este objeto ya no lo está usando nadie. Lo borro para liberar memoria”.

Así tú no tienes que borrar nada manualmente ni preocuparte por olvidarte de hacerlo.

Text blocks

Un text block (3 comillas dobles “””) es una forma nueva y cómoda de escribir texto largo en Java sin tener que poner comillas por todos lados ni usar \n para saltos de línea.

Antes, si querías escribir un texto largo, tenías que hacer algo así:

```
String html = "<html>\n" +  
    " <body>\n" +  
    "  Hola\n" +  
    " </body>\n" +  
    "</html>";
```

Con text blocks, lo escribes tal cual, como si fuera un texto normal:

```
String html = ""
```

```
<html>
```

```
<body>
```

```
  Hola
```

```
</body>
```

```
</html>
```

```
"";
```

Versión 16 (2021)

Elimina las opciones de compilación anticipada (y Graal JIT). La implementación de Java en sí estaba y sigue estando escrita en C++, mientras que a partir de Java 16 se permite una versión más reciente de C++14 (pero aún no, por ejemplo, C++17 o C++20). El código también se trasladó a GitHub, dejando de usar Mercurial como sistema de control de versiones.

- Vector API (Incubator)
- Enable C++14 Language Features
- Migrate to GitHub
- ZGC: Concurrent Thread-Stack Processing
- Unix-Domain Socket Channels
- Elastic Metaspace
- Foreign Linker API (Incubator)
- Warnings for Value-Based Classes
- Packaging Tool
- Foreign-Memory Access API (Third Incubator)
- Pattern Matching for instanceof
- Records
- Strongly Encapsulate JDK Internals by Default
- Sealed Classes (Second Preview)

Destaca: **Pattern Matching for instanceof, Records**

Pattern Matching for instanceof

Sirve para simplificar el uso de instanceof.

Antes, cuando comprobamos si un objeto era de un tipo concreto, tenías que:

1. Hacer el instanceof
2. Volver a castear el objeto manualmente

Con pattern matching, Java hace ambas cosas a la vez: Si el objeto es del tipo indicado, lo castea automáticamente y lo asigna a una variable.

Ejemplo de la forma antigua:

```
Object obj = "Hola mundo";
```

```
if (obj instanceof String) {  
    String s = (String) obj; // cast obligatorio  
    System.out.println(s.toUpperCase());  
}
```

```
}
```

Ejemplo usando pattern matching (forma moderna)

```
Object obj = "Hola mundo";

if (obj instanceof String s) {
    System.out.println(s.toUpperCase());
}
```

Records

Los records son un tipo especial de clase cuyo objetivo es representar datos inmutables de forma concisa, evitando escribir todo el “boilerplate” típico (constructores, getters, equals, hashCode, toString...).

Se usa porque Java llevaba años arrastrando un problema:
demasiado código repetitivo para clases que solo guardan datos.

Ejemplo clásico de clase “POJO”:

```
public class Persona {
    private final String nombre;
    private final int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() { return nombre; }
    public int getEdad() { return edad; }

    @Override
    public boolean equals(Object o) { ... }

    @Override
    public int hashCode() { ... }

    @Override
    public String toString() { ... }
}
```

Con un record, todo eso se reduce a:

```
public record Persona(String nombre, int edad) {}
```

Y Java genera automáticamente:

- Constructor
- Getters (llamados componentes)
- equals()
- hashCode()
- toString()

Versión 17 (2021)

Incluye las siguientes características nuevas, así como cientos de mejoras menores y miles de correcciones de errores.

- Restore Always-Strict Floating-Point Semantics
- Enhanced Pseudo-Random Number Generators
- New macOS Rendering Pipeline
- Deprecate the Applet API for Removal
- Strongly Encapsulate JDK Internals
- Pattern Matching for switch (Preview)
- Remove RMI Activation
- Sealed Classes
- Remove the Experimental AOT and JIT Compiler
- Deprecate the Security Manager for Removal
- Foreign Function & Memory API (Incubator)
- Vector API (Second Incubator)
- Context-Specific Deserialization Filters

Destaca: **sealed classes (clases selladas)**

Sealed Classes

Las sealed classes (clases selladas), permiten controlar explícitamente qué clases pueden heredar de otra. Esto le da a Java un control que antes solo existía de forma parcial con final (que prohíbe heredar) o con paquetes/módulos.

```
public sealed class Shape permits Circle, Rectangle, Square {
    public abstract double area();
}
```

```
public final class Circle extends Shape {
    private final double radius;
```

```
    public Circle(double radius) {
        this.radius = radius;
    }
```

```
    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}
```

```
public final class Rectangle extends Shape {  
    private final double width;  
    private final double height;  
  
    public Rectangle(double width, double height) {  
        this.width = width;  
        this.height = height;  
    }  
  
    @Override  
    public double area() {  
        return width * height;  
    }  
}
```

```
public non-sealed class Square extends Shape {  
    private final double side;  
  
    public Square(double side) {  
        this.side = side;  
    }  
  
    @Override  
    public double area() {  
        return side * side;  
    }  
}
```

Versión 18 (2022)

Incluye las siguientes características nuevas, así como cientos de mejoras menores y miles de correcciones de errores.

- UTF-8 by Default
- Simple Web Server
- Code Snippets in Java API Documentation
- Reimplement Core Reflection with Method Handles
- Vector API (Third Incubator)
- Internet-Address Resolution SPI
- Foreign Function & Memory API (Second Incubator)
- Pattern Matching for switch (Second Preview)
- Deprecate Finalization for Removal

Destaca: **Javadoc code snippets**

Javadoc code snippets

Son una forma moderna de incluir fragmentos de código dentro del Javadoc usando la etiqueta:

```
/**
 * Calcula el área de un círculo.
 *
 * {@snippet :
 * double r = 5;
 * double area = Math.PI * r * r;
 * System.out.println(area);
 * }
 */
public double area(double r) {
    return Math.PI * r * r;
}
```

Versión 19 (2022)

Permite que los patrones de registro, extendiendo las capacidades de coincidencia de patrones de los operadores `instanceof` y las expresiones `switch`, incluyan patrones de registro que se refieren explícitamente a los componentes del registro.

- Record Patterns (Preview)
- Linux/RISC-V Port
- Foreign Function & Memory API (Preview)
- Virtual Threads (Preview)
- Vector API (Fourth Incubator)
- Pattern Matching for `switch` (Third Preview)
- Structured Concurrency (Incubator)

Destaca: **Record Patterns (Preview)**

Record Patterns (Preview)

Esta fue la gran novedad que afectó directamente a los `record`. No cambia cómo se declaran, sino cómo se pueden usar para `pattern matching`.

Permiten desestructurar un `record` directamente en un `switch` o en un `if`, extrayendo sus componentes sin tener que llamar manualmente a los `getters`.

Versión 20 (2023)

Añade los `Virtual Threads` que permiten una gestión mucho más óptima de los temas de programación concurrente.

- Scoped Values (Incubator)
- Record Patterns (Second Preview)
- Pattern Matching for `switch` (Fourth Preview)
- Foreign Function & Memory API (Second Preview)
- Virtual Threads (Second Preview)
- Structured Concurrency (Second Incubator)
- Vector API (Fifth Incubator)

Versión 21 (2023)

La versión de 32 bits de Java para Windows en x86 quedó obsoleta y se eliminó con este lanzamiento. Se agregaron las siguientes JEP, incluidas ocho JEP que pasaron de las etapas de incubación y vista previa, en comparación con Java 20, que solo tenía JEP en vista previa e incubación. Java 21 introduce características que se presentaron por primera vez en Java 17 (coincidencia de patrones para sentencias switch) y Java 19 (patrones de registro). Todas las JEP agregadas con Java 21 incluyen las siguientes:

1. String Templates (Preview)
2. Sequenced Collections
3. Generational ZGC
4. Record Patterns
5. Pattern Matching for switch
6. Foreign Function & Memory API (Third Preview)
7. Unnamed Patterns and Variables (Preview)
8. Virtual Threads
9. Unnamed Classes and Instance Main Methods (Preview)
10. Scoped Values (Preview)
11. Vector API (Sixth Incubator)
12. Deprecate the Windows 32-bit x86 Port for Removal
13. Prepare to Disallow the Dynamic Loading of Agents
14. Key Encapsulation Mechanism API
15. Structured Concurrency (Preview)

Destaca: Record patterns, pattern matching for switch, **virtual threads**

Virtual Threads

Los Virtual Threads son hilos muy ligeros gestionados por la JVM, no por el sistema operativo. Su objetivo es permitir millones de hilos concurrentes sin el coste tradicional de los platform threads (los hilos clásicos de Java). Ejemplo: lanzar 1 millón de tareas

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 1_000_000; i++) {
        executor.submit(() -> {
            Thread.sleep(1000);
            return null;
        });
    }
}
```

Con hilos tradicionales esto sería imposible.
Con virtual threads... funciona sin problemas.

Mención especial: Versión 23 (2024)

Es una versión relativamente grande, con varias características en preview e incubación, pero con una novedad especialmente llamativa: los comentarios de documentación en Markdown.

Además, se retiró la preview de String Templates, porque el diseño no estaba funcionando como se esperaba.

1. Primitive Types in Patterns, instanceof, and switch (Preview)
2. Class-File API (Second Preview)
3. Markdown Documentation Comments
4. Vector API (Eighth Incubator)
5. Stream Gatherers (Second Preview)
6. Deprecate the Memory-Access Methods in sun.misc.Unsafe for Removal
7. ZGC: Generational Mode by Default
8. Module Import Declarations (Preview)
9. Implicitly Declared Classes and Instance Main Methods (Third Preview)
10. Structured Concurrency (Third Preview)
11. Scoped Values (Third Preview)
12. Flexible Constructor Bodies (Second Preview)

Destaca: **Markdown Documentation Comments**

Markdown Documentation Comments

Es una de las mejoras más útiles para desarrolladores. Hasta Java 22, la documentación de Javadoc se escribía con un formato propio, limitado y poco expresivo. Con Java 23, ahora puedes escribir comentarios Javadoc usando Markdown, lo que hace la documentación:

- más legible
- más fácil de escribir
- más familiar para la mayoría de desarrolladores
- más rica visualmente

Ejemplo antes (Javadoc tradicional)

```
/**
 * Returns the sum of two numbers.
 *
 * <p>This method adds {@code a} and {@code b}.</p>
 *
 * @param a first number
 * @param b second number
 * @return the sum
 */
```

Ejemplo ahora (Markdown)

```
/**
 * # Suma de dos números
 *
 * Este método añade a y b.
```

*

* - Parámetro `a`: primer número

* - Parámetro `b`: segundo número

*

* **Devuelve:** la suma de ambos.

*/